

# Library Inventory Manager – Mini Project Report

**Student Name:** Anshuman Sharma

**Course:** Programming for Problem Solving using Python

**Assignment Title:** Object-Oriented Design and Robust Programming in a Library Management System

This report documents the complete development of a Library Inventory Manager built using Python, demonstrating Object-Oriented Programming, JSON storage, exception handling, logging, and a user-friendly CLI system.

# 1. Project Overview

The Library Inventory Manager is a modular, lightweight system that allows users to manage book records easily.

It supports adding books, issuing books, returning books, and searching by title or ISBN.

The system uses:

- Object-Oriented Programming principles
- JSON-based file handling for persistent storage
- Robust exception handling for safety
- Logging to track important actions

Below is a screenshot showing the project's structure and testing commands:

```
1. Add Book
2. Issue Book
3. Return Book
4. View All
5. Search by Title
6. Search by ISBN
7. Exit
Choose an option (1-7): 4
- 'Harry Potter' by J.K. Rowling (ISBN: 12345) - available

Library Inventory Manager
1. Add Book
2. Issue Book
3. Return Book
4. View All
5. Search by Title
6. Search by ISBN
7. Exit
Choose an option (1-7): 7
Exiting. Bye.
(.venv) (base) savitasharma@Savita-Sharma library-inventory-manager-anshuman % python3 -m unittest discover -v

Ran 0 tests in 0.000s

NO TESTS RAN
(.venv) (base) savitasharma@Savita-Sharma library-inventory-manager-anshuman % ls -la tests
total 8
drwxr-xr-x 3 savitasharma staff 96 4 Dec 15:27 .
drwxr-xr-x 9 savitasharma staff 288 4 Dec 15:26 ..
-rw-r--r-- 1 savitasharma staff 875 4 Dec 15:27 test_inventory.py
(.venv) (base) savitasharma@Savita-Sharma library-inventory-manager-anshuman % sed -n '1,200p' tests/test_inventory.py

import unittest
from pathlib import Path
import tempfile
from library_manager.inventory import LibraryInventory
from library_manager.book import Book

class InventoryTest(unittest.TestCase):
    def setUp(self):
        self.tmpfile = Path(tempfile.mktemp(suffix=".json"))
        self.inv = LibraryInventory(self.tmpfile)

    def tearDown(self):
        try:
            self.tmpfile.unlink()
        except Exception:
            pass
```

Figure 1: Project structure and test execution output.

## 2. Implementation & CLI Demonstration

The project consists of three main modules:

- book.py** – Contains the Book class with attributes like title, author, ISBN, and status.
- inventory.py** – Contains the LibraryInventory class, managing catalog operations and JSON persistence.
- main.py** – Implements a menu-driven CLI for user interactions.

Below is a screenshot showing the CLI workflow for adding, issuing, returning, and viewing books:

```
Book added.

Library Inventory Manager
1. Add Book
2. Issue Book
3. Return Book
4. View All
5. Search by Title
6. Search by ISBN
7. Exit
Choose an option (1-7): 4
- 'Harry Potter' by J.K. Rowling (ISBN: 12345) - available

Library Inventory Manager
1. Add Book
2. Issue Book
3. Return Book
4. View All
5. Search by Title
6. Search by ISBN
7. Exit
Choose an option (1-7): 2
ISBN to issue: 12345
2025-12-04 15:50:14,839 [INFO] Saved 1 books to catalog.
Book issued.

Library Inventory Manager
1. Add Book
2. Issue Book
3. Return Book
4. View All
5. Search by Title
6. Search by ISBN
7. Exit
Choose an option (1-7): 4
- 'Harry Potter' by J.K. Rowling (ISBN: 12345) - issued

Library Inventory Manager
1. Add Book
2. Issue Book
3. Return Book
4. View All
5. Search by Title
6. Search by ISBN
7. Exit
Choose an option (1-7): 3
ISBN to return: 12345
2025-12-04 15:52:27,989 [INFO] Saved 1 books to catalog.
Book returned.

Library Inventory Manager
1. Add Book
```

Figure 2: CLI demonstrating book operations.

### 3. Summary & Conclusion

The Library Inventory Manager successfully demonstrates core Python programming concepts including:

- Class design and encapsulation
- Magic methods and data serialization
- Exception handling with try-except blocks
- Logging using Python's logging module
- Clean and modular project structure

The system is reliable, extensible, and can be expanded with additional features such as due date tracking, user accounts, or a graphical interface.

This project reflects strong understanding of both programming fundamentals and real-world application design.